Sutawane Classes Management System — Complete Development Plan

> **Document Status:** DRAFT — awaiting owner approval before any implementation begins. **Date:** 11 October 2026 **Prepared for:** Sutawane Classes (5–6 coaching branches, single-owner operation)

Table of Contents

- A. Existing Project Assessment
- B. Complete Feature Checklist
- C. Database and Data-Relationship Plan
- D. Business Rules
- E. Development Sequence
- F. Testing Plan
- G. Acceptance Criteria
- H. Final Verification Checklist
- I. Additional Essential Features
- J. Priorities

A. Existing Project Assessment

A.1 What Exists and Is Structurally Sound

Area	Status	Details
Monorepo structure	✅ Complete	npm workspaces with `frontend/`, `backend/`, `shared/`, `electron/` separation
React + Vite + TypeScript frontend	✅ Scaffolded	Vite 6.2, React 19, TypeScript 5.7 — project created via `npm create vite@latest`
Express + TypeScript backend	✅ Scaffolded	Express 4.21, tsx hot-reload, app factory pattern for testing
Prisma + SQLite database	✅ Scaffolded	Schema with 4 models, Prisma client configured, persistent file paths
Electron desktop shell	✅ Scaffolded	Main process, preload with `contextIsolation: true`, `nodeIntegration: false`
Shared types package	✅ Scaffolded	`@sutawane/shared` with Zod schemas and TypeScript interfaces
Persistent storage paths	✅ Complete	`config/index.ts` resolves `%APPDATA%/SutawaneClasses` in production; `data/dev` in development; creates `database/`, `photos/`, `attachments/`, `backups/` directories
Health check API	✅ Complete	`GET /api/health` — returns DB status, uptime, storage paths
Dependencies installed in package.json	✅ Listed	react-router-dom, @tanstack/react-query, react-hook-form, @hookform/resolvers, zod, recharts, lucide-react, tailwindcss, class-variance-authority, clsx, tailwind-merge, @playwright/test, vitest, @testing-library/*

A.2 What Exists but Requires Completion or Fixes

Area	Issue	Required Work
Tailwind CSS	Package installed but **not initialized** — no `tailwind.config.js`, no `postcss.config.js`, no `@tailwind` directives in CSS	Create config files, replace default Vite CSS with Tailwind setup
shadcn/ui	Utility packages installed (`cva`, `clsx`, `tailwind-merge`) but **no `components.json`**, no `cn()` utility, no component library	Run `npx shadcn@latest init`, configure path aliases
Vite config	Minimal — missing `resolve.alias` (`@` → `src`), missing API proxy to backend port 5000	Add alias + proxy configuration
TypeScript path aliases	`tsconfig.app.json` lacks `baseUrl` and `paths` for `@/*` imports	Add path mapping
React Router	Package installed but **not configured** — no `BrowserRouter`, no routes, no layouts	Set up routing with application shell
TanStack Query	Package installed but **not configured** — no `QueryClient`, no `QueryClientProvider`	Wire up in `main.tsx`
Frontend App.tsx	Still shows the default Vite counter demo	Replace with application shell
index.html title	Says `"frontend"`	Change to `"Sutawane Classes Management System"`
Student API	Basic `GET /` and `POST /` only — no `GET /:id`, `PUT /:id`, photo upload, pagination, filtering	Expand CRUD endpoints
Prisma schema	4 models (`Student`, `FeePayment`, `SystemSetting`, `HealthLog`) — flat structure, no branches, sessions, installments, expenses	Major schema redesign required
Student → FeePayment cascade	`onDelete: Cascade` will destroy financial records when a student is deleted	Change to soft-delete only; never hard-delete students with payments
Environment files	No `.env` files exist anywhere	Create `.env` templates
README.md	Only the default Vite template README in `frontend/` — no project-level README	Create comprehensive root README
Linting / Formatting	Only `oxlint` configured in frontend; no Prettier, no ESLint, no root formatting config	Add consistent formatting
npm install status	Previous install was interrupted/running	Verify and complete

A.3 What Is Completely Missing

Feature	Notes
Branch management (models, APIs, UI)	No `Branch` entity exists at all
Academic session management	No `AcademicSession` entity exists
Session-aware student admissions	`Student` has no session or branch foreign keys
Installment/fee-plan management	No `Installment` or `FeePlan` entities — only flat `FeePayment`
Expense tracking	No expense models or APIs
Dashboard with financial summaries	No dashboard page or components
Reports module	No report generation, CSV/XLSX/PDF export
Backup/restore functionality	Backup directory exists but no backup logic
Photo upload for students	`photoUrl` field exists but no upload endpoint or file handling
Receipt generation and printing	No receipt format or print logic
Any frontend pages or components	Zero application UI beyond the Vite demo
Any frontend API integration	No fetch calls, no API client
Database migrations	Prisma schema exists but no migration history
Git repository	No `.git` directory, no `.gitignore` at root

B. Complete Feature Checklist

B.1 Core Infrastructure (Priority: CRITICAL)

- **B.1.1** Complete Tailwind CSS + PostCSS initialization
- **B.1.2** Initialize shadcn/ui with `components.json` and `cn()` utility
- **B.1.3** Configure Vite path aliases (`@/`) and API proxy
- **B.1.4** Configure TypeScript path aliases in `tsconfig.app.json`
- **B.1.5** Set up React Router with `BrowserRouter` and layout shell
- **B.1.6** Set up TanStack Query `QueryClientProvider`
- **B.1.7** Create API client utility (`src/api/client.ts`) with base URL configuration

- **B.1.8** Create `.env` files for frontend (API URL) and backend (PORT, DATABASE_URL)
- **B.1.9** Replace Vite demo with application shell (sidebar, header, content area)
- **B.1.10** Set up root `.gitignore` and initialize git repository
- **B.1.11** Create root `README.md` with setup and run instructions
- **B.1.12** Verify `npm install` completes; verify `npm run dev` starts both servers
- **B.1.13** Run initial Prisma migration to create database

B.2 Branch Management (Priority: ESSENTIAL)

- **B.2.1** Create `Branch` Prisma model
- **B.2.2** Create branch CRUD API endpoints (`/api/branches`)
- **B.2.3** Build Branch list page with add/edit/activate/deactivate
- **B.2.4** Build global branch selector in the application header
- **B.2.5** Implement "All Branches" consolidated view mode
- **B.2.6** Pass selected `branchId` as filter parameter to all data-fetching APIs
- **B.2.7** Validate that every student, payment, and expense belongs to exactly one branch

B.3 Academic Session Management (Priority: ESSENTIAL)

- **B.3.1** Create `AcademicSession` Prisma model (name, startDate, endDate, isCurrent, isActive)
- **B.3.2** Create session CRUD API endpoints (`/api/sessions`)
- **B.3.3** Build Academic Session management page
- **B.3.4** Build global session selector in the application header
- **B.3.5** Ensure student admissions, fee plans, and installments are session-scoped
- **B.3.6** Implement controlled previous-session-dues recording
- **B.3.7** Preserve historical records across session transitions

B.4 Student Management (Priority: ESSENTIAL)

- **B.4.1** Redesign `Student` model as a permanent identity (no session-specific data)
- **B.4.2** Create `StudentAdmission` model linking Student to Branch + Session + Standard + Fees
- **B.4.3** Create complete student CRUD API: `GET /` , `GET /:id` , `POST /` , `PUT /:id` , `PATCH /:id/status`
- **B.4.4** Implement student photo upload endpoint and file storage
- **B.4.5** Implement search with filters (name, roll number, standard, branch, status)
- **B.4.6** Implement pagination for student lists

- **B.4.7** Build student list page with search, filter, pagination
- **B.4.8** Build student admission form (React Hook Form + Zod validation)
- **B.4.9** Build student profile/detail page with fee ledger summary
- **B.4.10** Build student edit page
- **B.4.11** Implement safe archiving (soft-delete, never hard-delete students with payment history)
- **B.4.12** Auto-generate unique student IDs (e.g., `SC-2026-0001`)

B.5 Installment Fee Management (Priority: ESSENTIAL — HIGHEST)

- **B.5.1** Create `FeePlan` model (total agreed fees, discount, net payable per student-admission)
- **B.5.2** Create `Installment` model (installment number, due date, amount due, status)
- **B.5.3** Create `Payment` model (amount, date, mode, receipt number, linked installment allocations)
- **B.5.4** Create `PaymentAllocation` model (junction: payment → installment, allocated amount)
- **B.5.5** Create `FeeDiscount` model (reason, amount, approved date)
- **B.5.6** Implement installment creation API (equal split or custom amounts)
- **B.5.7** Implement payment recording API with multi-installment allocation
- **B.5.8** Implement partial payment support
- **B.5.9** Implement sequential receipt number generation (e.g., `SC/2026-27/B1/REC-00001`)
- **B.5.10** Implement overpayment prevention (server-side validation)
- **B.5.11** Implement duplicate payment detection
- **B.5.12** Implement payment reversal/correction with audit trail
- **B.5.13** Build "Collect Fees" page — quick payment entry form
- **B.5.14** Build installment management page per student
- **B.5.15** Build payment history view
- **B.5.16** Build receipt view with print functionality
- **B.5.17** Build "Installments & Dues" page — upcoming, due, overdue, partial lists
- **B.5.18** Implement installment status auto-calculation (UPCOMING → DUE → OVERDUE)
- **B.5.19** Build student-wise fee ledger view

B.6 Expense Management (Priority: ESSENTIAL)

- **B.6.1** Create `ExpenseCategory` model with seed data (Rent, Salaries, Electricity, Internet, Stationery, Maintenance, Marketing, Miscellaneous)

- **B.6.2** Create `Expense` model (branch, date, category, amount, description, payment mode, vendor, reference number)
- **B.6.3** Create `ExpenseAttachment` model for bill scans/photos
- **B.6.4** Create expense CRUD API endpoints
- **B.6.5** Build expense entry form
- **B.6.6** Build expense list page with filters (branch, category, date range, payment mode)
- **B.6.7** Build expense edit with change history
- **B.6.8** Implement expense attachment upload

B.7 Dashboard (Priority: ESSENTIAL)

- **B.7.1** Create dashboard data aggregation API endpoints
- **B.7.2** Build dashboard page with today's summary cards (collections, expenses, net movement)
- **B.7.3** Build outstanding fees summary card
- **B.7.4** Build active student count card
- **B.7.5** Build due-today and overdue installment cards
- **B.7.6** Build monthly collection/expense bar chart (Recharts)
- **B.7.7** Build session-wise financial summary section
- **B.7.8** Build branch comparison section (when "All Branches" selected)
- **B.7.9** Build recent transactions list
- **B.7.10** Ensure all dashboard figures respect selected branch + session filters

B.8 Reports (Priority: ESSENTIAL)

- **B.8.1** Create report generation API endpoints with date-range, branch, session filters
- **B.8.2** Build report page shell with common filter controls
- **B.8.3** Daily collection report
- **B.8.4** Daily expense report
- **B.8.5** Daily financial movement report (collections − expenses)
- **B.8.6** Monthly collection report
- **B.8.7** Monthly expense report
- **B.8.8** Yearly collection and expense report
- **B.8.9** Academic-session-wise financial report
- **B.8.10** Branch-wise financial report
- **B.8.11** All-branches consolidated report

- **B.8.12** Student-wise fee ledger report
- **B.8.13** Outstanding and overdue installment report
- **B.8.14** Expense-category breakdown report
- **B.8.15** Payment-method breakdown report
- **B.8.16** Custom date-range report
- **B.8.17** Previous-session dues report
- **B.8.18** Academic-session comparison report
- **B.8.19** Implement CSV export for all reports
- **B.8.20** Implement XLSX export for all reports
- **B.8.21** Implement PDF export for key reports
- **B.8.22** Implement print-friendly report layouts

B.9 User Interface Shell (Priority: ESSENTIAL)

- **B.9.1** Design sidebar navigation with all modules
- **B.9.2** Build responsive header with branch selector + session selector
- **B.9.3** Build breadcrumb navigation
- **B.9.4** Install and configure shadcn/ui components: Button, Input, Select, Dialog, Table, Card, Tabs, Badge, DropdownMenu, Toast, AlertDialog, DatePicker, Popover, Sheet, Skeleton
- **B.9.5** Configure Lucide icons throughout
- **B.9.6** Implement loading states and skeleton screens
- **B.9.7** Implement error boundary and toast notifications
- **B.9.8** Implement confirmation dialogs for destructive actions
- **B.9.9** Ensure consistent form validation feedback (field-level errors)

B.10 Backup and Data Safety (Priority: ESSENTIAL)

- **B.10.1** Create backup API: copy SQLite file + photos + attachments to timestamped backup folder
- **B.10.2** Create restore API: validate backup, replace current database, restore files
- **B.10.3** Build backup management page (create backup, list backups, restore, download)
- **B.10.4** Implement automatic pre-operation backup before migrations and dangerous operations
- **B.10.5** Implement backup size and date display
- **B.10.6** Implement backup download (zip) for external storage

B.11 Settings (Priority: IMPORTANT)

- **B.11.1** Build settings page for institute name, logo, contact details, address
- **B.11.2** Store settings in `SystemSetting` table
- **B.11.3** Display institute details on receipts and printed reports
- **B.11.4** Configure receipt format preferences
- **B.11.5** Manage expense categories (add/edit/deactivate)

C. Database and Data-Relationship Plan

C.1 Entity-Relationship Diagram

C.2 Complete Entity Definitions

Branch

```
Branch
├── id            : String (UUID, PK)
├── name          : String (unique, e.g. "Nagpur Main", "Wardha Branch")
├── code          : String (unique, e.g. "NGP", "WRD")
├── address       : String? (optional)
├── phone         : String? (optional)
├── contactPerson : String? (optional)
├── isActive      : Boolean (default: true)
├── createdAt     : DateTime
└── updatedAt     : DateTime
```

AcademicSession

```
AcademicSession
├── id            : String (UUID, PK)
├── name          : String (unique, e.g. "2026-27")
├── displayName   : String (e.g. "Academic Year 2026–27")
├── startDate     : DateTime (e.g. June 2026)
├── endDate       : DateTime (e.g. May 2027)
├── isCurrent     : Boolean (only one session can be current at a time)
├── isActive      : Boolean (default: true)
├── createdAt     : DateTime
└── updatedAt     : DateTime
```

> **IMPORTANT**
>
> Academic sessions (e.g. June 2026 – May 2027) are distinct from Indian financial years (April 2026 – March 2027). Reports must support both date-range modes independently. Financial-year reporting is achieved by filtering on payment/expense dates, not by a separate financial-year entity.

Student (Permanent Identity)

```
Student
├── id            : String (UUID, PK)
├── studentCode   : String (unique, auto-generated: "SC-2026-0001")
├── fullName      : String
├── parentPhone   : String
├── studentPhone  : String?
├── address       : String?
├── photoUrl      : String? (relative path to photos directory)
├── dateOfBirth   : DateTime?
├── createdAt     : DateTime
└── updatedAt     : DateTime
```

> **NOTE**
>
> The `Student` entity stores only the permanent identity. All session-specific data (branch, standard, fees, active status) lives in `StudentAdmission`. This allows the same student to appear in multiple sessions without data duplication or loss.

StudentAdmission (Session-Scoped Enrollment)

```
StudentAdmission
├── id            : String (UUID, PK)
├── studentId     : FK → Student
├── branchId      : FK → Branch
├── sessionId     : FK → AcademicSession
├── rollNumber    : String (unique within branch+session)
├── standard      : String (e.g. "Class 10", "Class 12 Science")
├── admissionDate : DateTime
├── isActive      : Boolean (default: true — within this session)
├── createdAt     : DateTime
└── updatedAt     : DateTime
│
└── UNIQUE CONSTRAINT: (studentId, sessionId) — one admission per student per session
```

FeePlan (Per Admission)

```
FeePlan
├── id              : String (UUID, PK)
├── admissionId     : FK → StudentAdmission (unique — one fee plan per admission)
├── sessionId       : FK → AcademicSession
├── totalFees       : Float (agreed total)
├── totalDiscount   : Float (computed from FeeDiscount records)
├── netPayable      : Float (totalFees - totalDiscount)
├── totalPaid       : Float (computed — sum of payment allocations)
├── totalPending    : Float (computed — netPayable - totalPaid)
├── createdAt       : DateTime
└── updatedAt       : DateTime
```

> **TIP**
>
> `totalPaid` and `totalPending` are computed values derived from `PaymentAllocation` records. They should be recalculated on every payment or reversal rather than stored, **OR** stored as denormalized caches that are updated transactionally whenever a payment is recorded. The plan recommends the **computed approach** for correctness and uses backend service methods to calculate them on demand.

Installment

```
Installment
├── id              : String (UUID, PK)
├── feePlanId       : FK → FeePlan
├── installmentNo   : Int (1, 2, 3, ...)
├── label           : String (e.g. "Installment 1", "July Payment")
├── dueDate         : DateTime
├── amountDue       : Float
├── amountPaid      : Float (computed from PaymentAllocation)
├── status          : Enum (UPCOMING, DUE, OVERDUE, PARTIAL, PAID)
├── createdAt       : DateTime
└── updatedAt       : DateTime
```

Payment

```
Payment
├── id              : String (UUID, PK)
├── studentId       : FK → Student
├── branchId        : FK → Branch
├── receiptNumber   : String (unique, auto-generated: "SC/2026-27/NGP/REC-00001")
├── totalAmount     : Float (total cash/upi amount received)
├── paymentDate     : DateTime
├── paymentMode     : Enum (CASH, UPI, BANK_TRANSFER, CHEQUE, OTHER)
├── referenceNumber : String? (UPI ref, cheque number, etc.)
├── notes           : String?
├── isReversed      : Boolean (default: false)
├── reversalReason  : String?
├── reversedAt      : DateTime?
├── createdAt       : DateTime
└── updatedAt       : DateTime
```

PaymentAllocation (Junction: Payment ↔ Installment)

```
PaymentAllocation
├── id              : String (UUID, PK)
├── paymentId       : FK → Payment
├── installmentId   : FK → Installment
├── allocatedAmount : Float
├── createdAt       : DateTime
```

> IMPORTANT

> A single payment can be split across multiple installments (e.g., ₹5000 paid covering ₹3000 of Installment 2 and ₹2000 of Installment 3). The `PaymentAllocation` junction makes this possible while preserving an accurate audit trail.

FeeDiscount

```
FeeDiscount
├── id              : String (UUID, PK)
├── feePlanId       : FK → FeePlan
├── reason          : String (e.g. "Sibling discount", "Merit scholarship")
├── amount          : Float
├── approvedDate    : DateTime
├── notes           : String?
├── createdAt       : DateTime
```

PreviousSessionDue

```
PreviousSessionDue
├── id                    : String (UUID, PK)
├── studentId             : FK → Student
├── originalSessionId     : FK → AcademicSession (the session where the due originated)
├── currentSessionId      : FK → AcademicSession (the session where it's being collected)
├── outstandingAmount     : Float
├── collectedAmount       : Float (computed from linked payments)
├── description           : String
├── createdAt             : DateTime
└── updatedAt             : DateTime
```

> NOTE
>
> Previous-session dues are **not** automatically copied. The owner must explicitly record a previous-session balance for a student in the new session. Payments against these dues are tracked separately and reference the original session for reporting accuracy.

ExpenseCategory

```
ExpenseCategory
├── id        : String (UUID, PK)
├── name      : String (unique)
├── isActive  : Boolean (default: true)
├── sortOrder : Int
```

Expense

```
Expense
├── id              : String (UUID, PK)
├── branchId        : FK → Branch
├── categoryId      : FK → ExpenseCategory
├── amount          : Float
├── expenseDate     : DateTime
├── description     : String
├── paymentMode     : Enum (CASH, UPI, BANK_TRANSFER, CHEQUE, OTHER)
├── vendor          : String?
├── referenceNumber : String?
├── createdAt       : DateTime
└── updatedAt       : DateTime
```

ExpenseAttachment

```
ExpenseAttachment
├── id         : String (UUID, PK)
├── expenseId  : FK → Expense
├── fileName   : String
├── filePath   : String (relative path in attachments directory)
├── fileSize   : Int
├── mimeType   : String
├── createdAt  : DateTime
```

SystemSetting (Already Exists — Keep)

```
SystemSetting
├── key       : String (PK)
├── value     : String
└── updatedAt : DateTime
```

ReceiptCounter

```
ReceiptCounter
├── id        : String (UUID, PK)
├── branchId  : FK → Branch
├── sessionId : FK → AcademicSession
├── prefix    : String (e.g. "SC/2026-27/NGP/REC-")
├── lastNumber: Int (auto-incremented)
└── updatedAt : DateTime
│
└── UNIQUE CONSTRAINT: (branchId, sessionId)
```

C.3 Data Ownership Rules

Rule	Description
Branch isolation	Every `StudentAdmission`, `Payment`, and `Expense` must have a non-null `branchId`. Queries always filter by selected branch unless "All Branches" is active.
Session isolation	Every `StudentAdmission`, `FeePlan`, `Installment`, and `PreviousSessionDue` must have a session reference. Fee plans from one session never bleed into another.
Student permanence	The `Student` record is never deleted. Only `StudentAdmission.isActive` is toggled. A student's financial history across all sessions and branches is always accessible.
Payment immutability	Payments are never deleted. Corrections are handled by marking the original as `isReversed` and creating a corrective entry.
Expense audit	Expense edits create a log trail. The original amount and date are preserved in the edit history.
Receipt uniqueness	Receipt numbers are globally unique, auto-incremented per branch per session using `ReceiptCounter`.

D. Business Rules

D.1 Fee Calculation Rules

Rule	Logic
Net payable	`netPayable = totalFees - SUM(discounts.amount)`
Total paid	`totalPaid = SUM(paymentAllocations.allocatedAmount)` where `payment.isReversed = false`
Total pending	`totalPending = netPayable - totalPaid` (minimum 0)
Installment paid	`installment.amountPaid = SUM(paymentAllocations.allocatedAmount)` for that installment where payment is not reversed
Installment status	Computed based on date and payment: see table below

Installment Status Computation

```
IF installment.amountPaid >= installment.amountDue:
    status = PAID
ELSE IF installment.amountPaid > 0 AND installment.amountPaid < installment.amountDue:
    status = PARTIAL
ELSE IF today > installment.dueDate:
    status = OVERDUE
ELSE IF today = installment.dueDate:
    status = DUE
ELSE:
    status = UPCOMING
```

> IMPORTANT
>
> Installment status is **computed at read time**, not stored. This prevents stale data. The backend service method `getInstallmentStatus(installment)` calculates the correct status based on the current date and allocated payments every time it is requested.

D.2 Payment Allocation Rules

1. When recording a payment, the owner selects which installment(s) to apply it to.

2. The system suggests allocating to the **earliest unpaid or partially-paid installment first** (FIFO), but the owner can override.
3. A payment's `totalAmount` must equal the sum of its `PaymentAllocation.allocatedAmount` values.
4. An installment cannot receive allocations exceeding its `amountDue`.
5. If the payment amount exceeds all remaining installment balances, the system **rejects the payment** and asks the owner to verify. No silent overpayment.

D.3 Partial Payment Rules

1. A partial payment is any payment where `allocatedAmount < installment.amountDue`.
2. Multiple partial payments can be applied to the same installment.
3. The installment's status becomes `PARTIAL` until `amountPaid >= amountDue`.
4. The student ledger shows each partial payment with its date and allocated amount.

D.4 Payment Reversal Rules

1. A payment is never deleted from the database.
2. To correct a payment, the owner performs a **reversal**: `isReversed = true`, `reversalReason` recorded, `reversedAt` timestamped.
3. All `PaymentAllocation` records for the reversed payment are excluded from balance calculations.
4. A new corrective payment can then be recorded.
5. The audit trail shows: original payment → reversal → corrective payment.

D.5 Duplicate Payment Prevention

1. Before saving a payment, the backend checks:
    - Is there an existing non-reversed payment on the same date, same student, same amount, and same payment mode? → **Warning** (allow override).
    - Does the total of non-reversed payments for this student exceed `netPayable`? → **Rejection**.
2. The `receiptNumber` is unique and auto-generated, preventing exact duplicates.

D.6 Previous-Session Dues Rules

1. When a new academic session begins, outstanding balances from the previous session are **not** automatically carried forward.

2. The owner explicitly creates a `PreviousSessionDue` record for any student who owes from a prior session.
3. Payments against previous-session dues are recorded in the **current session** but tagged with `originalSessionId` so reports can distinguish between current-session collections and prior-session recoveries.
4. Previous-session dues do **not** inflate the current session's expected fee totals. They appear in separate report sections.

D.7 Expense Rules

1. Every expense is linked to exactly one branch.
2. Expense dates determine which reporting period (daily, monthly, yearly, financial year) they appear in.
3. Recurring expenses (e.g., monthly rent) are recorded as individual entries each month. There is no automatic recurrence — the owner enters each occurrence manually. This avoids complexity and maintains accuracy.
4. Expense categories can be deactivated but never deleted (preserving historical categorization).

D.8 Report Total Integrity Rules

Rule	Detail
No double-counting	Each payment is counted exactly once. Reversed payments are excluded. `PaymentAllocation` sums must equal the parent `Payment.totalAmount`.
Collections total	`SUM(payment.totalAmount)` where `isReversed = false` and `paymentDate` within the report period
Expenses total	`SUM(expense.amount)` where `expenseDate` within the report period
Net movement	`collections - expenses` for the period. This is **not** profit — it excludes opening balances, loans, and other non-fee income. The system does not track cash-in-hand.
Outstanding total	`SUM(feePlan.netPayable - feePlan.totalPaid)` across all active admissions in the selected session and branch
Financial year	For FY reporting, filter by April 1 through March 31, regardless of academic session boundaries
Cross-session	Session comparison reports query `FeePlan` and `Payment` tables filtered by respective session IDs

E. Development Sequence

The sequence is designed so that each step builds on the previous one. No step depends on a future step.

Phase 1: Foundation Fixes (Steps 1–6)

These fix and complete the existing project infrastructure. No new features — just making the existing scaffold work properly.

Step	Task	Depends On	Estimated Effort
1	Complete `npm install`, verify all workspaces resolve, fix any dependency issues	—	0.5 day
2	Initialize Tailwind CSS: create `tailwind.config.js`, `postcss.config.js`, replace `index.css` with Tailwind directives and base styles	Step 1	0.5 day
3	Initialize shadcn/ui: run `npx shadcn@latest init`, configure `components.json`, create `src/lib/utils.ts` with `cn()`, add path aliases to `vite.config.ts` and `tsconfig.app.json`	Step 2	0.5 day
4	Set up React Router: create `BrowserRouter` in `main.tsx`, create layout shell with sidebar and header, create placeholder pages, replace Vite demo	Step 3	1 day
5	Set up TanStack Query: create `QueryClient` and provider in `main.tsx`, create API client utility with base URL, configure Vite proxy for `/api` → `http://localhost:5000`	Step 4	0.5 day
6	Create `.env` files, root `.gitignore`, root `README.md`, verify `npm run dev` starts both frontend and backend, verify health check works from frontend	Step 5	0.5 day

Phase 2: Database Redesign (Steps 7–9)

The existing Prisma schema must be restructured before any feature development.

Step	Task	Depends On	Estimated Effort
7	Design complete Prisma schema: all entities from Section C (Branch, AcademicSession, Student, StudentAdmission, FeePlan, Installment, Payment, PaymentAllocation, FeeDiscount, PreviousSessionDue, ExpenseCategory, Expense, ExpenseAttachment, ReceiptCounter). Remove old flat `FeePayment` model.	Step 6	1 day
8	Run `prisma migrate dev` to create initial migration. Seed database with default expense categories.	Step 7	0.5 day
9	Create database service layer: Prisma client wrapper, transaction helpers, common query patterns (filter by branch, filter by session)	Step 8	1 day

Phase 3: Branch & Session Management (Steps 10–13)

These are prerequisites for everything else — all data is branch-scoped and session-scoped.

Step	Task	Depends On	Estimated Effort
10	Build Branch management API: CRUD endpoints, validation, uniqueness checks	Step 9	0.5 day
11	Build Branch management UI: list page, add/edit dialog, activate/deactivate, global branch selector in header	Step 10	1 day
12	Build Academic Session management API: CRUD, ensure only one `isCurrent` at a time	Step 9	0.5 day
13	Build Academic Session management UI: list page, add/edit dialog, activate/deactivate, global session selector in header, session-switching logic	Step 12	1 day

Phase 4: Student Management (Steps 14–18)

Step	Task	Depends On	Estimated Effort
14	Build Student + StudentAdmission APIs: create admission (with branch + session), get students by branch/session, search, filter, paginate	Steps 11, 13	1.5 days
15	Build Student photo upload API: accept image, resize, store in persistent photos directory, return URL	Step 14	0.5 day
16	Build Student list page: table with search, filter by standard/status, pagination, branch/session awareness	Step 14	1.5 days
17	Build Student admission form: React Hook Form + Zod, branch auto-set from selector, session auto-set	Step 16	1 day
18	Build Student profile/detail page: photo, info, admission history, fee summary, edit capability, archive action	Step 17	1 day

Phase 5: Installment Fee System (Steps 19–26) — HIGHEST PRIORITY FEATURE

Step	Task	Depends On	Estimated Effort
19	Build FeePlan + Installment creation API: set total fees, discounts, generate equal or custom installments	Step 14	1 day
20	Build Payment + PaymentAllocation API: record payment, validate allocation, generate receipt number, prevent overpayment and duplicates	Step 19	1.5 days
21	Build Payment reversal API: mark as reversed, recalculate installment balances	Step 20	0.5 day
22	Build fee plan & installment management UI: create/edit installments per student, status indicators	Step 19	1.5 days
23	Build "Collect Fees" page: student search → select → amount → payment mode → allocate to installments → confirm → receipt	Step 20	2 days
24	Build receipt view with print layout (receipt number, student details, amount, payment mode, date, institute details)	Step 23	1 day
25	Build payment history page: filter by student, date, mode; reversal action	Step 21	1 day
26	Build "Installments & Dues" page: upcoming, due today, overdue, partially paid — filterable by branch, session	Step 22	1 day

Phase 6: Expense Management (Steps 27–29)

Step	Task	Depends On	Estimated Effort
27	Build Expense CRUD API: create, edit with history, list with filters, attachment upload	Step 9	1 day
28	Build Expense management UI: entry form, list page with filters, edit dialog, attachment viewer	Step 27	1.5 days
29	Build Expense category management in Settings	Step 27	0.5 day

Phase 7: Dashboard (Steps 30–32)

Step	Task	Depends On	Estimated Effort
30	Build dashboard aggregation API: today's collections/expenses, monthly totals, outstanding fees, due counts, recent transactions	Steps 20, 27	1 day
31	Build dashboard UI: summary cards, Recharts bar/line charts, recent transactions list	Step 30	2 days
32	Add branch comparison view for "All Branches" mode	Step 31	0.5 day

Phase 8: Reports (Steps 33–37)

Step	Task	Depends On	Estimated Effort
33	Build report API endpoints: parameterized queries for all 16 report types with date range, branch, session filters	Steps 20, 27	2 days
34	Build report page shell: filter panel (date range picker, branch, session, category selectors), report table component	Step 33	1 day
35	Build all individual report views (daily, monthly, yearly, session-wise, branch-wise, student-wise ledger, outstanding, expense-category, payment-method, custom, previous-session, session-comparison)	Step 34	3 days
36	Implement CSV and XLSX export (using a library like `xlsx` or `exceljs` on the backend)	Step 35	1 day
37	Implement PDF export for receipts and key reports, print-friendly CSS layouts	Step 36	1.5 days

Phase 9: Backup & Safety (Steps 38–40)

Step	Task	Depends On	Estimated Effort
38	Build backup API: snapshot SQLite file + copy photos + attachments to timestamped backup folder, zip generation	Step 9	1 day
39	Build restore API: validate backup integrity, replace database, restore files with confirmation	Step 38	1 day
40	Build backup management UI: create backup button, backup list, restore button with confirmation, download zip	Step 39	1 day

Phase 10: Settings & Polish (Steps 41–44)

Step	Task	Depends On	Estimated Effort
41	Build Settings page: institute details, receipt preferences, expense category management	Step 29	1 day
42	Add PreviousSessionDue management: record previous-session dues for students, link payments	Step 20	1 day
43	End-to-end testing with Playwright: critical workflows (admission → fee plan → payment → receipt → report)	Step 37	2 days
44	Final polish: loading states, empty states, error messages, toast notifications, mobile responsiveness review	Step 43	1.5 days

Total Estimated Effort: ~47 development days

F. Testing Plan

F.1 Unit Tests (Vitest)

Test Area	Test Cases
Fee calculation service	Net payable calculation with 0, 1, multiple discounts; total paid with 0, 1, multiple payments; pending = net − paid; pending never goes below 0
Installment status computation	UPCOMING (future date, no payment), DUE (today, no payment), OVERDUE (past date, no payment), PARTIAL (past date, partial payment), PAID (any date, full payment)
Payment allocation validation	Sum of allocations = payment total; allocation cannot exceed installment due; reject overpayment
Receipt number generation	Sequential numbering, branch-session prefix, uniqueness
Duplicate payment detection	Same student + same date + same amount + same mode = warning
Zod validation schemas	Invalid phone numbers, empty names, negative fees, missing required fields

F.2 Integration Tests (Vitest + Supertest)

Test Area	Test Cases
Branch isolation	Create 2 branches, add students to each, verify GET students with branchId filter returns only that branch's students
Session isolation	Create 2 sessions, admit same student in both, create different fee plans, verify fee data doesn't cross sessions
Student admission	Create student → create admission → verify student appears in correct branch + session list
Full payment workflow	Create student → admission → fee plan → installments → record payment → verify allocations → verify installment status updates → verify receipt number generated
Partial payment	Record partial payment → verify installment status = PARTIAL → record remaining payment → verify status = PAID
Payment reversal	Record payment → reverse → verify installment balance reverts → verify reversed payment excluded from totals
Overpayment rejection	Create installments totaling ₹10,000 → attempt ₹15,000 payment → expect 400 error
Expense CRUD	Create expense → edit → verify history → filter by category and date
Report totals	Record known payments and expenses → verify daily/monthly report totals match
Backup/restore	Create data → take backup → delete all data → restore → verify all data restored

F.3 Frontend Component Tests (Vitest + React Testing Library)

Test Area	Test Cases
Branch selector	Render selector → select branch → verify context updates → verify API calls include branchId
Session selector	Render selector → select session → verify context updates
Student admission form	Fill valid data → submit → verify API call; fill invalid data → verify error messages
Payment form	Select student → enter amount → select installments → submit → verify allocation logic
Receipt print view	Render receipt with known data → verify all fields present

F.4 End-to-End Tests (Playwright)

Workflow	Steps
Complete admission-to-receipt	Navigate → create branch → create session → admit student → set fee plan → create installments → collect fee → verify receipt → check dashboard
Multi-branch isolation	Create 2 branches → add students to each → switch branch selector → verify only correct students shown
Report accuracy	Record known payments/expenses → generate daily report → verify totals match
Backup and restore	Create data → backup → clear data → restore → verify data intact
Previous-session dues	Create session 1 → admit student → leave balance → create session 2 → record previous-session due → collect payment → verify both sessions' reports

F.5 Database Persistence Tests

Test	Description
Restart persistence	Create records → stop server → start server → verify records exist
Migration safety	Run migrations → verify data preserved → run more migrations → verify again
Concurrent writes	Simulate 2 rapid payments for same student → verify no race conditions with receipt numbers

G. Acceptance Criteria

G.1 Branch Management Module ✓ Complete When:

- Can create, edit, activate, deactivate branches
- Global branch selector appears in the header on all pages
- "All Branches" shows consolidated data
- Selecting a specific branch filters all visible data
- Students, payments, and expenses from one branch never appear under another
- At least one branch must remain active

G.2 Academic Session Module ✓ Complete When:

- Can create, edit, activate, deactivate sessions
- Global session selector appears in the header on all pages
- Only one session can be marked as "current" at a time
- Student admissions, fee plans, and installments are scoped to the selected session
- Switching sessions does not alter or delete data from other sessions
- Previous sessions' data is read-only (cannot add new admissions to a closed session)

G.3 Student Management Module ✓ Complete When:

- Can admit a student with photo, name, roll number, standard, parent phone
- Student IDs are auto-generated and unique
- Roll numbers are unique within a branch + session
- Can search students by name, roll number, or phone
- Can filter by standard, active/inactive status
- Can view student profile with complete fee ledger
- Can edit student details and photo
- Can archive (deactivate) a student without losing their financial records
- Same student can be admitted in multiple sessions

G.4 Installment Fee Module ✓ Complete When:

- Can set total agreed fees and discounts for a student's admission
- Can create equal or custom-amount installments with due dates
- Can record a full payment against a single installment
- Can record a partial payment
- Can record a single payment split across multiple installments
- Receipt numbers are auto-generated, unique, sequential, and include branch/session prefix
- Can print/reprint a receipt
- Overpayments are rejected
- Duplicate payments trigger a warning
- Installment statuses (UPCOMING, DUE, OVERDUE, PARTIAL, PAID) are computed correctly
- Can reverse a payment with reason; financial totals update automatically
- Student fee ledger shows all payments, allocations, and balances

- "Collect Fees" page allows quick payment with under 5 clicks from start to receipt

G.5 Expense Module ✓ Complete When:

- Can record an expense with branch, date, category, amount, mode, description
- Can attach a bill scan or photo
- Can edit an expense; edit history is preserved
- Can filter expenses by branch, category, date range, payment mode
- Expense categories can be added and deactivated in Settings

G.6 Dashboard Module ✓ Complete When:

- Shows today's collections, today's expenses, today's net movement
- Shows total outstanding fees
- Shows active student count
- Shows installments due today and overdue count
- Shows monthly collection/expense chart
- Shows recent transactions list
- All figures update correctly when branch or session selector changes
- "All Branches" mode shows branch comparison data

G.7 Reports Module ✓ Complete When:

- All 16 report types generate correctly
- Report totals match the sum of underlying transactions (spot-check with manual calculation)
- Reversed payments are excluded from totals
- Financial-year filtering (Apr–Mar) works independently of academic sessions
- CSV export produces valid downloadable files
- XLSX export produces valid downloadable files
- PDF export produces printable documents for key reports
- Print layouts render properly in the browser print dialog

G.8 Backup Module ✓ Complete When:

- Can create a full backup (database + photos + attachments) with one click
- Backup files are timestamped and stored in the persistent backups directory
- Can download a backup as a zip file

- Can restore from a backup with confirmation dialog
- After restore, all data matches the backup state
- Backup works when the application is packaged as an Electron desktop app

H. Final Verification Checklist

Before the software is used for actual financial records, verify:

H.1 Data Integrity

- Create 3 branches, 2 academic sessions, 20 students spread across branches and sessions
- Record 50 payments with various modes (cash, UPI, cheque, bank transfer)
- Record 20 expenses across categories
- Verify all dashboard totals match manual calculations
- Verify daily, monthly, and yearly report totals match manual calculations
- Verify branch-filtered reports exclude other branches' data
- Verify session-filtered reports exclude other sessions' data
- Record a partial payment, verify installment shows PARTIAL status
- Record remaining payment, verify installment shows PAID status
- Reverse a payment, verify financial totals decrease correctly
- Verify no student can accidentally be double-charged

H.2 Data Safety

- Create a backup → close application → reopen → verify data intact
- Create a backup → deliberately corrupt database → restore from backup → verify data restored
- Verify database file is in persistent `%APPDATA%` location, not in build or temp folders
- Verify photos and attachments are stored outside build folders
- Try to delete a student with payment history → verify soft-archive only, financial records preserved
- Verify Electron is configured with `contextIsolation: true` and `nodeIntegration: false`

H.3 Usability

- Complete a new student admission in under 2 minutes
- Collect a fee and print a receipt in under 1 minute

- Record an expense in under 1 minute
- Find a specific student using search in under 10 seconds
- Generate and print a daily collection report in under 30 seconds
- Switch branches and verify the interface updates immediately
- All form validation errors are clearly displayed in Hindi or English
- Confirmation dialogs appear before any destructive action (delete, reverse, restore)

H.4 Technical

- `npm run dev` starts both frontend and backend without errors
- Frontend loads at `http://localhost:5173` and connects to backend
- Backend API responds at `http://localhost:5000/api/health`
- `npm run build` produces production builds for frontend and backend
- Electron desktop app loads the production frontend and connects to backend
- All automated tests pass (`npm test`, `npm run test:frontend`, `npm run test:e2e`)

I. Additional Essential Features

The following features were not explicitly mentioned in the requirements but are necessary for a reliable, daily-use coaching management system:

I.1 Toast Notifications (ESSENTIAL)

Why: The owner needs immediate visual feedback after every action — saving a student, recording a payment, creating a backup, etc. Without toast notifications, the owner won't know if an action succeeded or failed.

Implementation: Use shadcn/ui's Toast component. Show success (green), error (red), and info (blue) toasts for all create/update/delete operations.

I.2 Confirmation Dialogs (ESSENTIAL)

Why: Accidental clicks on "Reverse Payment," "Deactivate Student," or "Restore Backup" can cause data issues. Every destructive action must require explicit confirmation.

Implementation: Use shadcn/ui's AlertDialog with clear messaging: "Are you sure you want to reverse this payment of ₹5,000 for Rahul Sharma? This action will update his outstanding balance."

I.3 Empty States (ESSENTIAL)

Why: When the software is first used or when a branch has no students, the owner should see helpful guidance — not a blank white page.

Implementation: Every list page shows an illustrated empty state with a call-to-action button (e.g., "No students yet. Click 'Add Student' to get started.").

I.4 Loading Skeletons (ESSENTIAL)

Why: API calls take time. Without loading indicators, the owner will think the application is broken and click buttons repeatedly.

Implementation: Use shadcn/ui's Skeleton component for table rows, cards, and charts during data loading.

I.5 Date Formatting for Indian Locale (ESSENTIAL)

Why: The owner expects dates in DD/MM/YYYY format and currency in ₹ format. Using US date formats (MM/DD/YYYY) will cause confusion and errors.

Implementation: Use `Intl.DateTimeFormat('en-IN')` and `Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR' })` consistently throughout the application.

I.6 Keyboard Shortcuts for Daily Tasks (IMPORTANT)

Why: The owner will collect fees dozens of times daily. Quick keyboard shortcuts (e.g., `Ctrl+P` for new payment, `Ctrl+S` for save) improve speed.

Implementation: Add shortcuts for the most common actions in Phase 10 polish.

I.7 Database Auto-Backup Before Migrations (ESSENTIAL)

Why: If a Prisma migration fails or corrupts the schema, the owner could lose all data. An automatic backup before any migration protects against this.

Implementation: The migration script automatically copies the SQLite file to `backups/` with a `pre-migration-` prefix before running `prisma migrate deploy`.

I.8 Data Export for External Accountant (IMPORTANT)

Why: Sutawane Classes likely has an external accountant who needs monthly or yearly financial summaries. The accountant expects Excel files, not a login to the software.

Implementation: XLSX export with proper column headers, formatted currency, and date ranges — available from every report page.

I.9 Audit Log for Financial Transactions (IMPORTANT)

Why: If a dispute arises about a payment or refund, the owner needs to trace exactly when it was recorded, modified, or reversed, and what the values were.

Implementation: A lightweight `AuditLog` table recording `action`, `entityType`, `entityId`, `oldValue`, `newValue`, `timestamp` for all payment and expense operations.

J. Priorities

J.1 Essential — Required for First Usable Release

These must be completed before the software can replace manual record-keeping:

Priority	Module	Steps
P0	Foundation fixes (Tailwind, shadcn, Router, Query, .env)	Steps 1–6
P0	Database redesign with all required entities	Steps 7–9
P0	Branch management (API + UI + global selector)	Steps 10–11
P0	Academic Session management (API + UI + global selector)	Steps 12–13
P0	Student management (admission, list, profile, search)	Steps 14–18
P0	Installment fee system (HIGHEST PRIORITY: fee plans, installments, payments, receipts, ledger)	Steps 19–26
P0	Expense management (entry, list, categories)	Steps 27–29
P0	Dashboard (today's summary, outstanding, charts)	Steps 30–32
P0	Daily and monthly reports with print	Steps 33–35 (subset)
P0	Backup with one-click create + restore	Steps 38–40
P0	Settings (institute details for receipts)	Step 41
P0	Toast notifications, confirmation dialogs, empty states, loading skeletons	Step 44

J.2 Important — Should Be Completed Shortly After First Release

Priority	Feature	Steps
P1	Full 16-type report suite	Steps 33–35 (remaining)
P1	CSV and XLSX export for all reports	Step 36
P1	PDF export for receipts and key reports	Step 37
P1	Previous-session dues management	Step 42
P1	Academic session comparison report	Part of Step 35
P1	Expense attachment upload	Part of Step 27
P1	Audit log for financial transactions	New step
P1	Data export for external accountant	Part of Step 36

J.3 Optional — Future Enhancements

Priority	Feature	Notes
P2	Electron desktop packaging with installer	`electron-builder` config exists; needs testing and signing
P2	Auto-backup on schedule (daily/weekly)	Cron-style backup within Electron
P2	Keyboard shortcuts for daily tasks	Quality-of-life improvement
P2	SMS/WhatsApp fee reminders	Requires third-party API integration
P2	Student attendance tracking	Not in current requirements; mentioned as a future possibility
P2	Staff/teacher management	Not in current requirements
P2	Multi-user login with roles	Owner-only software currently; add later if branch managers are hired
P2	Student/batch/timetable management	Advanced scheduling features
P2	Mobile-responsive progressive web app (PWA)	For on-the-go access
P2	Cloud backup to Google Drive or OneDrive	External storage redundancy
P2	Dark/light theme toggle	Aesthetic preference

> CAUTION
>
> **This plan is ready for review.** No code has been written, no files modified, and no commands executed. Development will begin only after your explicit approval. Please review each section and note any changes, additions, or corrections you'd like before we proceed.